CodeDNA v0.9 WHITE PAPER

An In-Source Communication Protocol for AI Coding Agents

How embedding architectural context directly in source files improves LLM agent file localization F1 — +17pp on DeepSeek (10 tasks, p=0.001)

Author: Fabrizio Corpora

Date: April 2026 (v0.9 · updates March 2026 v0.7)

License: MIT (Open Source)

Repository: github.com/Larens94/codedna

EXECUTIVE SUMMARY

AI coding agents — tools like GitHub Copilot, Claude Code, and Cursor — are increasingly deployed on real, multi-file production codebases. The dominant failure mode is not reasoning quality: it is **navigation cost**. Agents spend most of their token budget finding the right files, and when they misidentify the source of a bug, they introduce regressions instead of fixes.

CodeDNA is a lightweight, language-agnostic in-source communication protocol that solves this by embedding a navigable dependency graph directly inside every source file — no external systems, no retrieval latency, no build step. A multi-model SWE-bench evaluation now shows the strongest result on **DeepSeek Chat: +17.1pp mean F1 across 10 Django tasks** (Wilcoxon exact p=0.0009765625, wins 10/0/0), with 6 of the 10 tasks independently replicated by an external contributor. **+13pp on Gemini 2.5 Flash** (p=0.040) and **+9pp on Gemini 2.5 Pro** confirm the gain across model families. Version 0.9 adds three new fields — `related:`, `wiki:`, and `message:` — plus a tooling layer that converts annotations into an Obsidian knowledge vault. The standard is open source (MIT), integrated with the major AI coding tools, and available at github.com/Larens94/codedna.

+17pp	3/3	52%→25%	0
file localization F1 on DeepSeek Chat · 10 tasks · p=0.001 · 10/0/0	models improved on SWE-bench multi-model evaluation	high-risk run rate reduced across SWE-bench models	external dependencies — context travels with the code

1. The Problem: AI Agents Are Navigation-Blind

Modern AI coding assistants are powerful reasoners but poor navigators. SWE-bench (2024), the leading multi-file agent benchmark, found that state-of-the-art models resolved only **1.96% of real GitHub issues**. The bottleneck was not code generation quality — it was the cost of finding the right files in the first place.

❌ CURRENT APPROACHES FAIL BECAUSE...	✅ CODEDNA WORKS BECAUSE...

- **RAG systems** require embedding infrastructure and introduce retrieval latency that may not align with architectural relevance
- **Rule files** (CLAUDE.md, AGENTS.md, .cursorrules) are invisible to agents in sliding-window mode — the agent reading line 50 of a file cannot see the project-level rules file
- **Long-context injection** inflates inference costs and degrades with distance
- A 2026 ETH Zurich study found LLM-generated context files *degrade* task success by 3% and increase costs by 20%+

- Context is **co-located with the code** — it travels in every file read, in every context window, at every sliding position
- No external systems: zero retrieval latency, zero infrastructure, zero sync issues
- Uses **native comment and docstring syntax** — no new toolchain required
- Scales with the codebase: the more files, the larger the navigation advantage

> *"When an agent reads only lines 50–80 of a file, it encounters function-level annotations and inline comments that reassert the critical architectural rules exactly at the point of risk." — CodeDNA Specification v0.9*

2. How CodeDNA Works: The Biological Analogy

The name *CodeDNA* captures the core design principle through a molecular biology analogy. Biological DNA encodes the complete blueprint of the organism in *every cell*. No matter which cell a biologist examines, it carries the full genome. CodeDNA applies the same principle to software: every source file encodes not only its own logic, but also its *position in the architectural organism* — what it depends on, who depends on it, and what contracts it must respect.

LEVEL 0 Project Manifes — **Repo-level map** — a single `.codedna` YAML file at the repo root. The agent reads this first — before opening any source file — to understand package purposes and inter-package dependencies. Injected into the agent's first message so it has the full structural map before making any tool call.

LEVEL 1 Module Header — **File-level manifest** — placed at the top of every source file. Encodes the complete architectural position in ~70 tokens: dependencies, exports, consumers, database tables touched, and agent-facing rules. Visible even when the file is read from the very first line.

LEVEL 2 Sliding Window — **Function-level annotations** — docstrings and inline call-site comments. Agents often extract 50–100 lines from the middle of a file, bypassing the module header. Level 2 re-asserts dependency contracts at every cross-file function call.

LEVEL 3 Semantic Nami — **Variable naming convention** — format: `<type>_<shape>_<domain>_<origin>`. Any 10-line context window containing a variable name carries its type, origin, and semantic role without requiring a lookup.

```
"""analytics/revenue.py — Monthly revenue aggregation.

deps:    payments/models.py → get_invoices_for_period
         tenants/models.py → is_suspended
exports: monthly_revenue(year, month) → dict
used_by: api/reports.py → revenue_route
rules:   get_invoices_for_period() returns ALL tenants, NO suspension filter
         → MUST call is_suspended() BEFORE aggregating revenue
"""
```

Planner Read Protocol

For agents planning edits across 10+ files, CodeDNA enables a structured read protocol: (1) read `.codedna` first to get the full package map; (2) read only the module docstring of each relevant file (first 8–12 lines); (3) build an `exports:` → `used_by:` navigation graph; (4) open only the files where the graph intersects with the task. This pattern reduces unnecessary file reads while maximising coverage of architecturally relevant files — the same mechanism responsible for CodeDNA's improved precision in the benchmarks.

3. Who Benefits

🏢

Engineering Teams

AI coding assistants introduce fewer regressions in your codebase. Every PR touched by Copilot or Claude Code benefits from embedded navigation context.

🧰

AI Tool Vendors

Integration files already exist for Claude Code, Cursor, GitHub Copilot, and Windsurf. CodeDNA is a drop-in protocol that improves agent outcomes without model changes.

🔬

Researchers

A reproducible benchmark framework for measuring LLM navigation efficiency. Both codebase sets are open source and deterministic at temperature 0.

4. Adoption: Zero-Cost Start

CodeDNA requires no new toolchain. Annotations are plain comments and docstrings in whichever comment syntax the language already uses. An agent that writes CodeDNA annotations as it generates code pays zero marginal annotation cost — the architectural memory is a natural byproduct of code generation.

For existing codebases, the CLI tool `tools/codedna_setup.py` installs AI integration files in under 60 seconds. A validation tool (`tools/validate_manifests.py`) checks annotation completeness as a CI step. GitHub Actions configuration is provided out of the box.

5. SWE-bench Multi-Model Evaluation (v0.9)

CodeDNA has been evaluated on real Django issues from **SWE-bench** — a public benchmark of real GitHub issues. Each task was run 3–5 times per condition at T=0.1 across 3 LLMs (max 30 turns). Statistical significance is evaluated with the Wilcoxon signed-rank test (one-tailed). **The DeepSeek condition was extended in April 2026 from 5 to 10 tasks**, with 6 of the new tasks **independently replicated** by an external contributor (@fabioscialanga) on a separate machine.

Model	Tasks	Ctrl F1	DNA F1	Δ F1	p-value	Tasks Won
DeepSeek Chat	10	50.9%	**68.0%**	**+17.1pp**	**0.001** ✅✅	**10 / 10**
Gemini 2.5 Flash	5	60%	**72%**	**+13pp**	0.040 ✅	4 / 5
Gemini 2.5 Pro	5	60%	**69%**	**+9pp**	0.11	3 / 5

DeepSeek Chat (10 tasks, W+=55, N=10) achieves strong statistical significance at **p=0.0009765625** (Wilcoxon exact, one-tailed), with CodeDNA winning on **all 10 tasks** — no inversions, no ties. Gemini 2.5 Flash (5 tasks, W+=14, N=5): p=0.040, Δ=+13pp. Gemini 2.5 Pro (5 tasks, W+=12, N=5): Δ=+9pp, p=0.11. Two DeepSeek tasks (11808 and 13121) exhibit a stability pattern: CodeDNA standard deviation across 3

runs is **0.00** (identical result every time) vs control std 0.20–0.25 — the advantage is not just a higher mean, it is a reduction in run-to-run unpredictability.

> **Independent replication.** 6 of the 10 DeepSeek tasks (13121, 15629, 16263, 11400, 11883, 11808) were executed independently by @fabioscialanga on a separate machine with the same runner, prompts, and task definitions. All 10 tasks favour CodeDNA. Raw session traces are in `labs/benchmark/runs/deepseek-chat/session_traces/`. Independent replication on heterogeneous hardware with matching protocol is the strongest evidence the current sample size permits.

5.1 When CodeDNA Helps Most

Analysis across 5 tasks (Gemini 2.5 Flash, ≥5 runs each) suggests a pattern correlated with task type:

Task type	Example	Δ F1 (Flash)
Clear dependency chain — A calls B which delegates to C	`dbshell → client → subprocess` (12508)	**+9%**
Delegation with backend fan-out — one interface, N backends	`Trunc → ops.date_trunc_sql` (13495)	**+22%**
Feature addition with flag gating — across feature/schema layers	`INCLUDE clause in Index` (11991)	**+17%**
XOR feature with multi-layer propagation	`Q() XOR support` (14480)	**+17%**
Cross-cutting fix — same pattern in N unrelated files, no shared ancestor	`__eq__ NotImplemented` (11808) · IDNA domains (11532)	v0.7: ≈0% · **v0.9 `related:`: 11532 40%→100%; 11808 +25.5pp on DeepSeek 10-task**

Rows 1–4: Gemini 2.5 Flash, v0.7 protocol, `used_by:` only.

With `used_by:` alone, CodeDNA is most effective when there is a navigable call chain. The `used_by:` graph guides the agent from entry point to all affected files. For cross-cutting concerns (same fix across N independent files with no common ancestor), the benefit of `used_by:` alone is near zero because no navigation path connects them. This was the structural limit of v0.7 — reported transparently rather than hidden. **v0.9's `related:` field closes this gap:** on Django bug #11532 (5-file IDNA fix with no shared import ancestor), adding `related:` brings F1 from 40% to 100%; on task 11808 under the expanded 10-task DeepSeek condition, Δ=+25.5pp.

Per-task qualitative analysis. Task 12508 (dbshell): the entry point is obvious by name; the challenge is discovering four backend client files — `used_by:` handles this reliably (+9pp). **Task 11991** (INCLUDE): `schema.py` is findable by grep; `used_by:` enumerates all four backend schema editors (+17pp). **Task 14480** (Q() XOR): a multi-layer ORM→SQL→backends chain requires touching 7 files — exactly what `used_by:` enables (+17–18pp). **Task 13495** (Trunc tzinfo): the most model-divergent task. Flash gains +22pp; DeepSeek and Pro regress by 8–9pp. Following `used_by:` chains to backend files appears to introduce a precision trap for less capable models — a structural anomaly under investigation. **Task 11808** (`__eq__`): identified in v0.7 as the structural limit of the protocol — the entry point `models/base.py` (847 lines) is a generic file with no navigable chain to the five affected subclasses. v0.9 addresses this class of bug through the `related:` field (see §6), which lets a file point to its semantic siblings outside the import graph. On Django bug #11532 (IDNA domain handling, a 5-file cross-cutting fix), adding `related:` brings file-localization F1 from 40% to

100%. Under the expanded DeepSeek 10-task condition, task 11808 now shows Δ=+25.5pp — a clean ablation on the contribution of `related:` alone remains open.

5.2 The Cheaper-Model Hypothesis

The empirical data supports a working hypothesis:

> ***Less capable, cheaper models benefit more from CodeDNA.*** *A frontier model navigates large codebases well by general reasoning. A cheaper model without structural guidance gets lost, loops, or stops early. CodeDNA provides the scaffolding that lets a cheap model approach the navigation quality of a more expensive one.*

Evidence: DeepSeek Chat (Δ=+17.1pp, p=0.001, 10 tasks), Gemini 2.5 Flash (Δ=+13pp, p=0.040), Gemini 2.5 Pro (Δ=+9pp). DeepSeek shows the strongest absolute gain and highest statistical significance. Notably, Pro ctrl=60% matches Flash — Pro is not stronger than Flash on these navigation tasks, suggesting that navigation efficiency does not scale linearly with model size. This makes CodeDNA economically attractive: *annotate once, run cheaper models with comparable accuracy*.

Note: this hypothesis is supported by the DeepSeek result (p=0.001, highly significant, 10 tasks) and the Flash result (p=0.040, statistically significant). Gemini 2.5 Pro shows a positive but non-significant trend (p=0.11). A larger multi-model study across more task types is required to confirm or refute this hypothesis.

5.3 Annotation Integrity Audit

A key concern with any navigation benchmark is whether the annotations guide the agent to the right files by describing the architecture, or by encoding the solution. We performed a full audit of the benchmark annotations to verify that no task-specific hints were embedded.

Methodology: For each task, every ground-truth file (from `files_in_patch`) was inspected. The `used_by:` targets were classified as GT (ground-truth) or non-GT. The `rules:` field was checked for task-specific prescriptions vs. architectural descriptions.

Task	`used_by:` verdict	`rules:` verdict
11808 (`__eq__`)	Files are independent — no `used_by:` chain exists by design	Describes Python data model convention, not a file list. v0.7 Δ≈0% with `used_by:` alone confirms no advantage from the annotation; v0.9 `related:` adds Δ=+25.5pp (DeepSeek 10-task)
14480 (XOR)	16 targets in `query_utils.py`; only 2 are GT, both genuine callers	Describes connector system mechanics; agent still needs to reason which files to open
13495 (Trunc tzinfo)	`base/operations.py` lists all 4 backends — they are all the backends that exist	`_convert_field_to_tz()` mentioned per-backend: accurate architecture, not a fix prescription
11991 (INCLUDE)	`base/schema.py` corrected during audit to include all 4 backend schema editors	Architectural delegation pattern; no file list for the fix
12508 (dbshell)	`base/client.py` lists all 4 backend clients — complete by definition	`dbshell.py` rules describe existing architecture accurately

Verdict: the benchmark annotations are architecturally accurate and do not encode task-specific solutions. Where GT files appear in `used_by:` targets, it is because those files are genuine callers or subclasses. The empirical results are consistent with this: tasks with navigable chains show Δ>0%, the cross-cutting task (11808) shows Δ≈0% with `used_by:` alone — confirming the `rules:` text does not encode the solution. Note: v0.9's `related:` field addresses this class of bug and shows Δ=+25.5pp on DeepSeek (10-task).

Annotation correction made during audit: `django/db/backends/base/schema.py` in task 11991 had an incomplete `used_by:` listing only `postgresql/schema.py`. All four backend schema editors genuinely inherit from `BaseDatabaseSchemaEditor`. The annotation was updated to reflect the complete inheritance graph before benchmark runs.

Note: this audit was conducted by the paper's authors on their own benchmark annotations. Independent verification on externally annotated codebases remains future work.

5.4 Retry Cost Analysis

File-localization F1 is typically framed as an efficiency metric. We argue it should also be treated as a **correctness proxy**: an agent that reads fewer than half the ground-truth files is unlikely to produce a complete patch, forcing the user to re-invoke the entire session. We define a run as *high-risk* when per-run recall falls below 50%.

Across 131 multi-run trials (DeepSeek-Chat, Gemini 2.5 Flash, Gemini 2.5 Pro — 5 tasks each, 3–6 runs per task):

Condition	High-risk runs	Total runs	Risk rate
Control	34	66	**52%**
CodeDNA	16	65	**25%**
Reduction			**–27 pp (–52% relative)**

The most pronounced case is DeepSeek-Chat on task 11808 (10 ground-truth files): all five control runs achieved recall of 10–20%; all five CodeDNA runs achieved 30–60% recall. Every control run would have produced an incomplete patch. Gemini 2.5 Pro provides a complementary data point: CodeDNA reduced mean tool calls by 14.3% while raising mean F1 by 9 pp — fewer calls, higher accuracy simultaneously.

These results support an **accuracy-first framing** for annotation protocols: even if CodeDNA were cost-neutral in tool calls, halving the rate of incomplete navigation would justify its adoption by halving the rate of sessions the user must repeat.

6. Research Roadmap and v0.9 Protocol Additions

The current SWE-bench evaluation establishes the proof of concept across 3 LLMs, with the strongest result on DeepSeek Chat (10 tasks, p=0.001, 10/10). Version 0.9 introduces three new optional fields that complement the v0.7 four-level architecture:

- **`related:`** — semantic cross-cutting links. While `used_by:` captures *structural* dependencies (imports), `related:` captures files that *share a pattern* without importing each other. On Django bug #11532 (5-file

IDNA fix with no import ancestor) F1 jumps from 40% → 100% when `related:` is added. Addresses the task-11808 class of cross-cutting bugs identified as the structural limit of v0.7.

- **`wiki:`** — opt-in pointer from a docstring to a curated markdown page under `docs/wiki/`. Sparsity is the signal: the pointer exists only when a prior agent decided the terse header was insufficient. Companion tooling (`codedna wiki bootstrap` and `codedna wiki sync`) turns the annotation graph into an Obsidian-ready knowledge vault plus a Karpathy-style narrative project wiki — both regenerable from a post-commit hook.
- **`message:`** — inter-agent conversational layer inside the `agent:` field. Append-only, with promote-to-`rules:` or dismiss-with-reason lifecycle. In the AgentHub multi-agent SaaS experiment (DeepSeek R1, 5 agents, 83 min), `message:` was adopted spontaneously in 54 of 55 annotated files (98.2%) with no mid-session reminders.

Further planned extensions that have not yet been empirically evaluated:

- **Extended benchmark coverage** — bring Flash and Pro to 10 tasks each, add a placebo condition (CodeDNA-shaped header with plausible-but-false annotations), extend to end-to-end patch correctness
- **LLM-assisted `related:` generation** — a single LLM pass per file to suggest semantic siblings from the full corpus, with agent veto
- **Project-level `cross_cutting_patterns:`** — a Level 0 manifest field complementing the file-level `related:`
- **Multi-language adapters** — already shipped for PHP, TypeScript, Go, Java, Kotlin, Ruby, Rust, C# via tree-sitter; more real-world testing on non-Python projects
- **Peer-reviewed publication** — MSR 2026 or ICSE 2027 submission

6.1 CodeDNA as a Self-Generating Training Corpus

The following section describes a speculative future direction with no current empirical validation.

A consequential property of the protocol is that a CodeDNA-annotated codebase with active agent usage constitutes a **self-generating training corpus** across three complementary supervision levels:

SFT (Supervised Fine-Tuning). The append-only `agent:` field accumulates an ordered, file-scoped log of correct navigational decisions. Each line encodes what an agent did, what constraint it discovered, and what it left for the next session. Across thousands of files and sessions, this is a dense demonstration dataset of expert codebase navigation grounded in real task outcomes — zero labelling cost.

DPO / Preference Alignment. The git history with AI trailers (see §6.2) produces naturally occurring *(rejected, chosen)* pairs. When Agent A introduces an incorrect annotation and Agent B corrects it and updates `rules:`, the two commits form a labelled preference pair — complete with visited-file lists, session IDs, and agent-written rationale — produced at zero marginal cost during normal development.

PRM (Process Reward Model). The session trace infrastructure records the full ordered sequence of tool calls per session. Combined with binary task outcome, each trace becomes a labelled reasoning trajectory, enabling a process-level reward model trained on real agent behaviour.

These three levels are mutually reinforcing — a **data flywheel**: better models → more accurate annotations → more informative preference pairs → better process reward models → sharper navigational policy → better models. Each agent session that writes or corrects a CodeDNA annotation improves the training signal

available to all future model generations without additional human labelling effort. *The codebase is simultaneously the environment, the protocol, and the training dataset.*

6.2 Further Protocol Extensions

Agent telemetry via git trailers. A proposed extension embeds AI session metadata directly in commit messages using *git trailers* — the same convention as `Co-Authored-By:`, natively recognised by GitHub. Proposed fields: `AI-Agent:`, `AI-Provider:`, `AI-Session:`, `AI-Visited:` (files read during the session, which git does not track natively), and `AI-Message:`. Combined with the existing `.codedna` session log and the file-level `agent:` field, this creates a *three-tier audit architecture* linked by a shared `session_id`: git (authoritative diff and metadata), `.codedna` (navigable session summary), and file `agent:` (sliding-window-safe one-liner). Already in use in the CodeDNA repository itself as of v0.9.

Verification agents. The v0.9 introduction of `related:`, `wiki:`, and `message:` expands the surface of annotations that can drift from the code. *Verification agents* would periodically cross-check annotations against the actual codebase — validating that `used_by:` and `related:` links resolve to existing files, `rules:` constraints are still respected, `wiki:` pointers are non-broken, and `message:` observations have been resolved. Because annotations are structured and machine-readable, this maintenance is itself fully automatable.

Try CodeDNA Today

Open source · MIT License · Ready-to-use integrations for Claude Code, Cursor, GitHub Copilot, Windsurf

⭐ GitHub Repository 📖 Documentation ✉ Contact